

Advanced JUnit Testing Exercises

Exercise 1: Parameterized Tests

Create EvenChecker.java

```
public class EvenChecker {  
  
    public boolean isEven(int number) {  
        return number % 2 == 0;  
    }  
}
```

Create EvenCheckerTest.java

```
import static org.junit.jupiter.api.Assertions.assertEquals;  
import org.junit.jupiter.params.ParameterizedTest;  
import org.junit.jupiter.params.provider.ValueSource;  
  
public class EvenCheckerTest {  
  
    EvenChecker checker = new EvenChecker();  
  
    @ParameterizedTest  
    @ValueSource(ints = {2, 4, 6, 8, 10})  
    void testEvenNumbers(int number) {  
        assertEquals(true, checker.isEven(number));  
    }  
  
    @ParameterizedTest  
    @ValueSource(ints = {1, 3, 5, 7, 9})  
    void testOddNumbers(int number) {  
        assertEquals(false, checker.isEven(number));  
    }  
}
```

Exercise 2: Test Suites

Create CalculatorTests.java

```
import static org.junit.jupiter.api.Assertions.*;  
import org.junit.jupiter.api.Test;  
  
public class CalculatorTests {
```

```
@Test
void additionTest() {
    assertEquals(10, 5 + 5);
}
```

```
@Test
void subtractionTest() {
    assertEquals(5, 10 - 5);
}
}
```

Create StringTests.java

```
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;
```

```
public class StringTests {

    @Test
    void testLength() {
        assertEquals(5, "Hello".length());
    }

    @Test
    void testContains() {
        assertTrue("Java".contains("av"));
    }
}
```

Create AllTests.java

```
import org.junit.platform.suite.api.SelectClasses;
import org.junit.platform.suite.api.Suite;
```

```
@Suite
@SelectClasses({
    CalculatorTests.class,
    StringTests.class
})
public class AllTests {
}
```

Exercise 3: Test Execution Order

Create OrderedTests.java

```
import org.junit.jupiter.api.Order;
import org.junit.jupiter.api.Test;
import org.junit.jupiter.api.TestMethodOrder;
import org.junit.jupiter.api.MethodOrderer;

@TestMethodOrder(MethodOrderer.OrderAnnotation.class)
public class OrderedTests {

    @Test
    @Order(1)
    void firstTest() {
        System.out.println("First Test");
    }

    @Test
    @Order(2)
    void secondTest() {
        System.out.println("Second Test");
    }

    @Test
    @Order(3)
    void thirdTest() {
        System.out.println("Third Test");
    }
}
```

Exercise 4: Exception Testing

Create ExceptionThrower.java

```
public class ExceptionThrower {

    public void throwException() {
        throw new ArithmeticException("Division by zero");
    }
}
```

Create ExceptionThrowerTest.java

```
import static org.junit.jupiter.api.Assertions.assertThrows;

import org.junit.jupiter.api.Test;

public class ExceptionThrowerTest {

    ExceptionThrower obj = new ExceptionThrower();

    @Test
    void testException() {

        assertThrows(
            ArithmeticException.class,
            () -> obj.throwException()
        );
    }
}
```

Exercise 5: Timeout and Performance Testing

Create PerformanceTester.java

```
public class PerformanceTester {

    public void performTask() throws InterruptedException {
        Thread.sleep(1000);
    }
}
```

Create PerformanceTesterTest.java

```
import static org.junit.jupiter.api.Assertions.assertTimeout;

import java.time.Duration;

import org.junit.jupiter.api.Test;

public class PerformanceTesterTest {

    PerformanceTester tester = new PerformanceTester();

    @Test
    void testPerformance() {
```

```

assertTimeout(
    Duration.ofSeconds(2),
    () -> tester.performTask()
);
}
}

```









